

Abstract

VYUHA is a controlled defensive validation framework for testing whether EDR products detect, block, or miss cross-layer evasion behavior.

It combines four modules: BYOVD, EDR-Freeze, Crystal Palace reflective loading, and SysWhispers4 direct syscall execution.

A DQN selector learns product-specific action choices, while SHAP, clustering, and transfer learning explain defensive patterns.

Problem Statement

Modern EDR products observe many endpoint layers, but their detection chain can break at driver trust, process availability, memory inspection, or API-hook visibility.

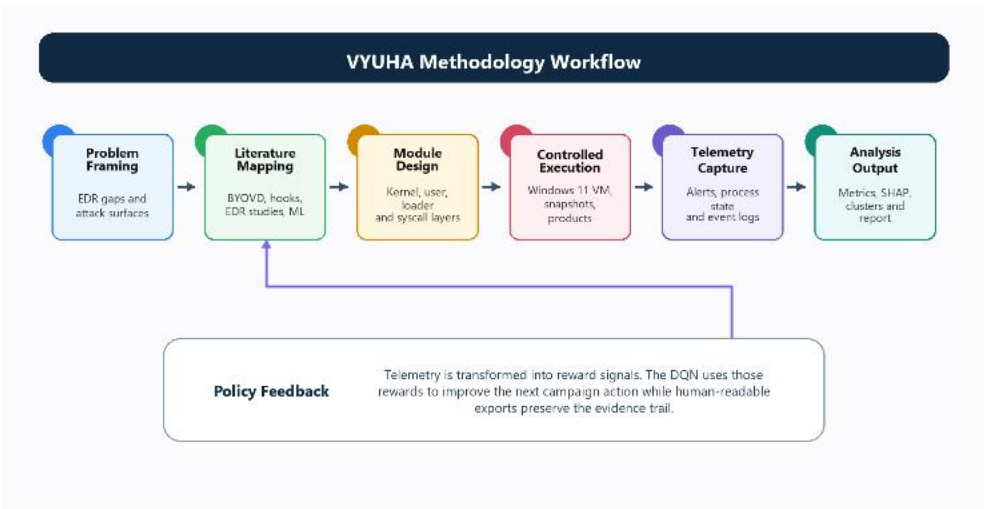
Existing evaluations often test fixed techniques manually, so they miss how products behave when attack sequencing adapts to previous responses.

Objectives

- Build a modular C++17 framework for controlled EDR evaluation.
- Execute four ATT&CK-mapped techniques across kernel, process, memory, and syscall layers.
- Record success, detection, blocking, time-to-detection, cleanup, and export evidence.
- Use DQN selection plus SHAP, clustering, and transfer learning for explainable comparison.

Methodology

Each run starts from a restored Windows 11 VM snapshot. VYUHA installs one EDR profile, dispatches a selected module, captures alerts, process state, timing, and cleanup status, then computes metrics and reward features. The ML layer uses the same result schema to recommend the next action.



VYUHA Technique-to-Defense Map			
How each implemented module addresses a different detection assumption			
Technique	Primary layer tested	Expected telemetry	Why it matters
BYOVD (17042)	Kernel/Process/Driver	Driver load, OCL process, process state	Tests if driver-based evasion is blocked (e.g., blocked exports)
EDR-Freeze (17042)	Process/Service/Process	APIs, kernel, process state, agent heartbeat loss	Tests if the EDR can survive process state changes
Crystal Palace (17042)	Memory/Kernel/Process	APIs, kernel, process state, agent heartbeat loss	Tests if the EDR can survive memory state changes
SysWhispers4 (17042)	User mode/Kernel/Process	APIs, kernel, process state, agent heartbeat loss	Tests if the EDR can survive user mode state changes
DQN selector	User mode/Kernel/Process	APIs, kernel, process state, agent heartbeat loss	Tests if the EDR can survive user mode state changes

Outcomes: the framework reports not only whether detection occurred, but which defensive assumption failed



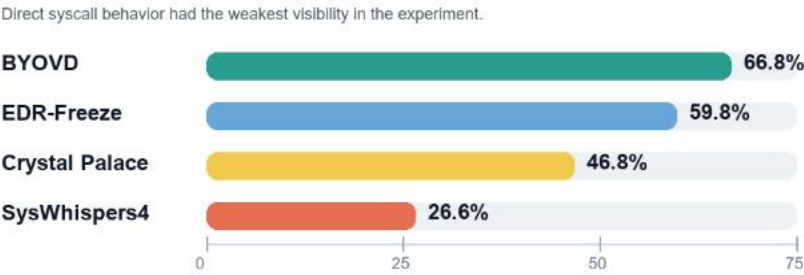
Results



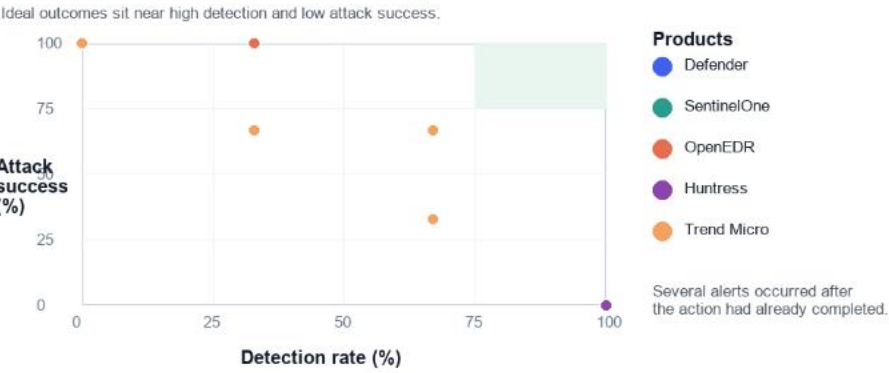
Detection coverage by product and technique



Mean detection rate by technique

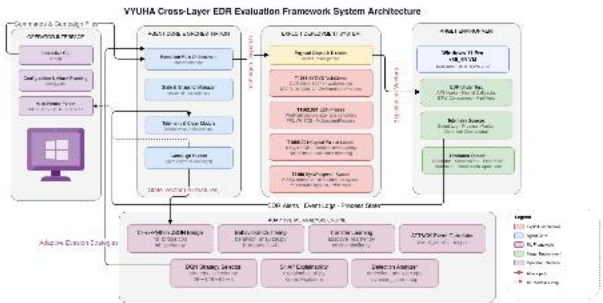


Detection is not the same as prevention



Key Findings

- SentinelOne and Huntress blocked BYOVD and EDR-Freeze in every run.
- Crystal Palace and SysWhispers4 still created partial bypasses in stronger products.
- OpenEDR had the weakest coverage, with several successful runs and no visible alert.
- SysWhispers4 had the lowest mean detection rate, showing that user-mode hook coverage is incomplete.



References

Karantzas & Patsakis (2021), Empirical assessment of EDR systems against APT vectors.
MDSec Research (2020), Bypassing user-mode hooks and direct invocation of system calls.
Haag, Bradley & Hernandez (2025), Living Off The Land Drivers.
Mnih et al. (2013), Playing Atari with Deep Reinforcement Learning.
Lundberg & Lee (2017), SHAP: A unified approach to interpreting model predictions.
Pan & Yang (2010), A survey on transfer learning.



Report QR